1. 什么是 **Kafka**？它通常用来解决什么问题？**What is Kafka, and what problems does it solve?**
Apache Kafka is an open-source distributed event streaming platform. It receives continuous streams of events, stores them durably, and delivers them in real time to the systems that need them.
Kafka mainly solves the problem of integrating many systems. Without it, every system has to connect directly to every other system, creating a tangled web of integrations with different protocols and data formats. With Kafka, producers publish data to Kafka and consumers read from it, so the systems are decoupled. Common uses include asynchronous communication between microservices, real-time data pipelines, log collection, and stream processing.

2. 什么是消息（**Message/Record**）？一条 **Kafka** 消息通常可以包含哪些信息？**What is a message (record), and what information can a Kafka message contain?**
A message, also called a record, is the basic unit of data in Kafka and represents a single event, such as an order being created. Producers write messages and consumers read them.
A message usually contains a key (optional, used to choose the partition, so messages with the same key stay in order), a value (the actual content), optional headers for metadata such as a trace ID, and a timestamp. It can also be compressed. Once written, it gets a partition number and an offset, and topic, partition, and offset together uniquely identify the message. Keys and values are stored as bytes, so they are serialized before sending and deserialized when read.

3. 什么是生产者（**Producer**）？它在 **Kafka** 中承担什么职责？ **What is a producer, and what is its role in Kafka?**
A producer is an application that writes messages to Kafka topics. It creates each message, serializes its key and value into bytes, and decides which partition it goes to: messages with the same key are sent to the same partition, while messages without a key are spread across partitions. The producer sends the message to the partition's leader broker and uses the acks setting to decide when a write counts as successful, retrying if it fails. For efficiency, producers also batch and compress messages.

4. 什么是消费者（**Consumer**）？它在 **Kafka** 中承担什么职责？**What is a consumer, and what is its role in Kafka?**
A consumer is an application that reads messages from Kafka topics. It uses a pull model, calling poll() to request messages so it can control its own pace. It deserializes the messages, processes them in offset order within each partition, and commits its offset to record how far it has read, so it can resume after a crash. Consumers usually work in a consumer group, where the group's partitions are divided among its members and each partition is read by only one consumer in the group.

5. 什么是主题（**Topic**）？为什么 **Kafka** 要使用 **Topic** 对消息进行分类？**What is a topic, and why does Kafka use topics to categorize messages?**
A topic is a named category of messages, similar to a table in a database, such as orders or logs. A topic is made up of one or more partitions where the messages are actually stored.
Topics keep different kinds of data separate, so consumers can subscribe only to the data they need. Each topic can also be configured independently, for example with its own number of partitions, replication factor, and retention period.

6. 什么是 **Broker**？**Kafka** 集群中为什么可以有多个 **Broker**？**What is a broker, and why can a Kafka cluster have multiple brokers?**
A broker is a single Kafka server. It stores partition data on disk, handles read and write requests from producers and consumers, and replicates data to other brokers. A group of brokers working together forms a Kafka cluster, and each broker has a unique numeric ID.

A cluster has multiple brokers for scalability and fault tolerance. A topic's partitions are spread across brokers, so the cluster can store more data and handle more traffic than one server. Each partition is also replicated on several brokers, so if one broker fails, another replica becomes the leader and no data is lost.

7. 什么是分区（**Partition**）？一个 **Topic** 为什么可能需要多个 **Partition**？**What is a partition, and why might a topic need multiple partitions?**
A partition is one part of a topic: an ordered, append-only log of messages that cannot be changed once written. Partitions are numbered from 0 to N-1.
Multiple partitions allow a topic to scale and to be processed in parallel. Partitions can be placed on different brokers, so a topic is not limited by one server, and consumers in the same group can each read a different partition at the same time. The trade-off is that order is only guaranteed within a partition, so messages that must stay in order should use the same key.

8. 什么是 **Offset**？它表示的是消息的全局编号，还是消息在某个 **Partition** 中的位置？**What is an offset? Is it a global message number or a position within a partition?**
An offset is the position of a message within a specific partition, not a global number. Offsets start at 0 in every partition, so offset 3 in partition 0 and offset 3 in partition 1 are different messages. A message is uniquely identified by its topic, partition, and offset. Consumers also use offsets to record how far they have read.

9. 什么是消费者组（**Consumr Group**）？同一个消费者组中的多个消费者如何分工？ **What is a consumer group, and how do consumers in the same group share the work?**
A consumer group is a set of consumers that work together on the same task, identified by the same group.id. The partitions of a topic are divided among the consumers in the group: each partition is assigned to only one consumer in the group, while one consumer can handle several partitions. This lets the group process data in parallel without reading the same message twice.

10. 什么是消息键（**Key**）？**Key** 会如何影响消息被写入哪个 **Partition**？**What is a message key, and how does it affect which partition a message is written to?**
A key is an optional field in a message, such as an order ID or user ID, used to decide which partition the message goes to. If the key is null, messages are spread evenly across partitions in a round-robin way, with no ordering guarantee between them. If a key is provided, Kafka hashes it (murmur2 by default) and takes the result modulo the number of partitions. The same key always maps to the same partition, so all messages with that key are stored and read in order. For example, using the order ID as the key keeps the created, paid, and shipped events of one order in sequence.

11. Topic 和 **Partition** 有什么区别？请用一句话描述它们之间的关系。**What is the difference between a topic and a partition?**
A topic is a logical category of messages identified by a name, while a partition is a physical, ordered, append-only log where the messages are actually stored. Order is guaranteed within a partition but not across a whole topic, and each partition has its own offsets.

12. Broker 和 **Kafka** 集群有什么区别？一个 **Broker** 是否就等于一个 **Kafka** 集群？**What is the difference between a broker and a Kafka cluster? Is one broker the same as a Kafka cluster?**
A broker is a single Kafka server, while a Kafka cluster is a group of brokers working together to store and serve data.

A broker is not generally the same as a cluster; it is one member of it. A cluster can technically run with a single broker, which is common for development, but it then has no fault tolerance or horizontal scalability, so production clusters use multiple brokers.

13. Producer 和 Consumer 是否需要彼此知道对方是谁？请说明原因。Do producers and consumers need to know about each other? Explain why.
No. Producers only write to a topic and consumers only read from a topic, so neither needs to know who the other is. They only need to agree on the topic name and the data format.
This decoupling means new consumers can be added without changing producers, both sides can work at their own pace, and one side can be offline without affecting the other, since messages stay in Kafka until consumers read them.

14. Offset 是由 Producer 决定的吗？消费者为什么需要记录自己的消费位置？Is the offset decided by the producer? Why do consumers need to record their position?
No. The offset is assigned by the partition leader broker when the message is appended. The producer only chooses the topic and partition.
Kafka does not delete messages after they are read, so each consumer group must track its own position. The committed offset lets a consumer resume after a crash or rebalance, lets different groups read independently, and allows data to be replayed.

15. "Kafka 能保证整个 Topic 中的所有消息严格有序。"这句话是否正确？请说明 Kafka 的顺序保证范围。"Kafka guarantees that all messages in a topic are strictly ordered." Is this correct? Explain the scope of Kafka's ordering guarantee.
No. Kafka only guarantees order within a single partition, not across a topic. To keep related messages in order, give them the same key so they go to the same partition. Ordering an entire topic requires a single partition, which gives up parallelism.

16. 如果一个 Topic 有 3 个 Partition，同一个消费者组中有 5 个 Consumer，那么最多有几个 Consumer 能同时消费这个Topic？其余 Consumer 会怎样？If a topic has 3 partitions and a consumer group has 5 consumers, how many consumers can consume at the same time? What happens to the others?
At most 3 consumers can consume at the same time, because each partition can be assigned to only one consumer in a group. The other 2 consumers stay idle. They act as standbys: if an active consumer fails, Kafka rebalances and assigns its partition to an idle consumer, which resumes from the last committed offset. To increase parallelism, the number of partitions must be increased.

17. 两个不同的消费者组订阅同一个 Topic，它们会互相抢消息吗？请解释它们各自能看到哪些消息。If two different consumer groups subscribe to the same topic, do they compete for messages? Which messages does each group see?
No. Each consumer group receives all messages in the topic independently, because each group tracks its own offsets and Kafka does not delete messages after they are read. Within a group, partitions are divided among its consumers, but across groups there is no competition. For example, an inventory service and a notification service can both read every order event.

Optional:
Play with Transactional Outbox Demo following Runbook.md in Github repo. Understand Faliure Matrix in the tutorial.